



## 2. Git. GitHub. Погружаемся

[Jump to bottom](#)

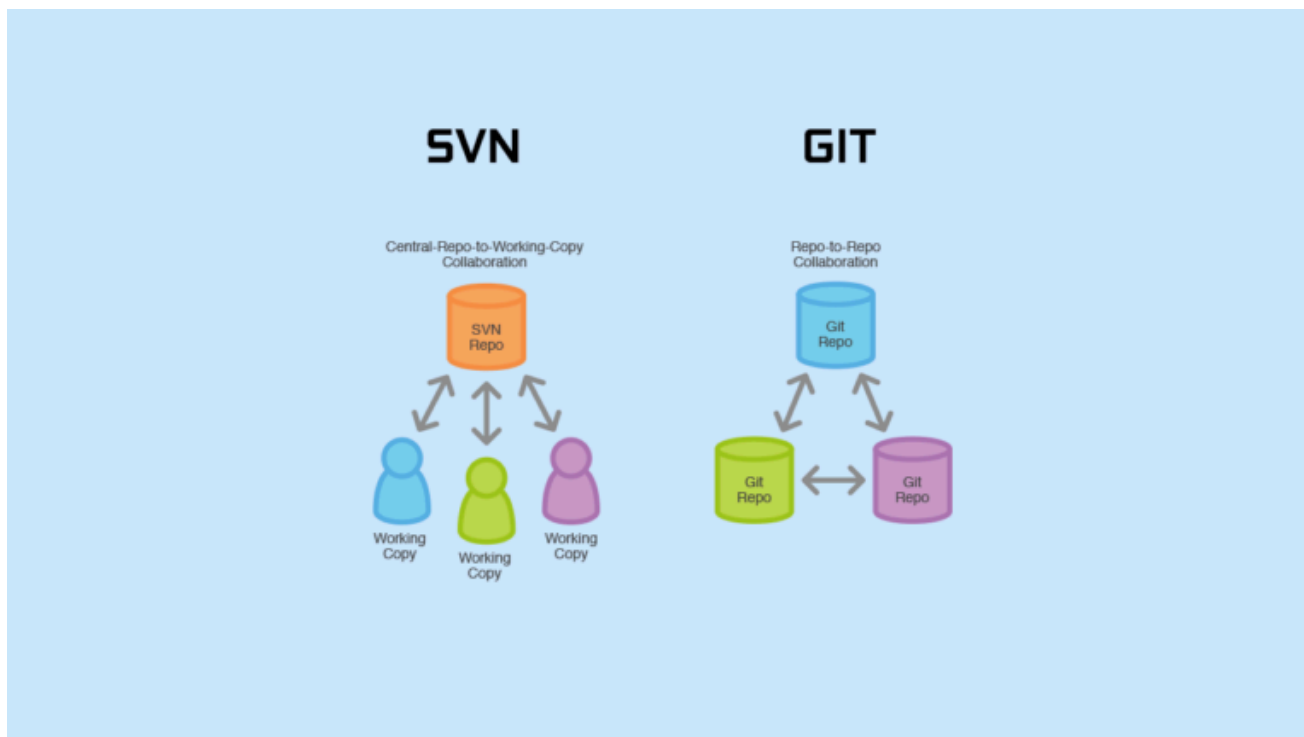
Nikita Starkov edited this page on Feb 16 · [6 revisions](#)

### Оглавление

1. [Справка о Git](#)
2. [Справка о GitHub](#)
3. [Генерация SSH ключа](#)
4. [Добавление SSH ключа в GitHub](#)
5. [Создание репозитория](#)
6. [Алгоритм работы с Git/Github](#)
7. [Работа с Git из IntelliJ IDEA](#)
8. [Справка про .gitignore](#)

## Git

Git — система управления версиями с распределенной архитектурой. Можно считать лидером в области ПО для контроля версий. Главным отличием от некогда популярных CVS и SVN является именно распределенная архитектура. Если раньше единственная версия проекта и вся история изменений хранилась исключительно в одном месте, то с приходом Git каждая рабочая версия стала полноценным репозиторием. Это позволяет каждому участнику проекта локально хранить всю историю изменений и код.



Предполагается, что на вашей машине уже установлен `git`, вы уже попробовали клонировать репозиторий и выполнили первое домашнее задание. Если нет, то по этой [ссылке](#) можно посмотреть инструкцию по установке `git`, а по [этой](#) по клонированию репозитория и запуску первого теста.

#### 💡 Tip

Маленькая [шпаргалка](#) по основным командам Git от команды GitHub Education.

## GitHub

### Знакомство с сервисом

[GitHub](#) — один из самых крупных и популярных сервисов для хостинга IT-проектов и совместной работы над ними. GitHub построен на базе Git, а значит поддерживает всего его функции. Кроме основных функций в GitHub есть и другие, добавленные разработчиками сервиса: контроль доступа к коду, базы знаний, управление задачами, интеграцию с другими сервисами, запросы на принятия изменений и другие. Кроме GitHub есть и другие сервисы онлайн-хостинга репозитория: GitLab и Bitbucket.

Интерфейс `github` выглядит следующим образом:

#### ► Скриншоты (нажать, чтобы раскрыть)

На первом скриншоте:

1. **Блок с информацией о пользователе:** аватар, имя пользователя, никнейм, поле «О себе», количество подписчиков и подписок, место работы, локация, адрес электронной почты, ссылка на сайт, значки сообщества и список организаций.
2. **Панель переключения вкладок:**
  - Overview — просмотр главной страницы профиля (то, что на скриншоте);
  - Repositories — список репозиториях пользователя;
  - Projects — своеобразный планировщик задач, привязанный к GitHub. Пока доступен в бете;
  - Packages — вкладка с пакетами, опубликованными пользователем;
  - Stars — репозитории, отмеченные «звездочкой». В GitHub можно ставить «звездочки» чужим репозиториям. К примеру, если нашли интересный проект и хотите позже предложить разработчикам свои идеи или просто установить, то можно отметить его «звездочкой», тогда он появится в этой вкладке.
3. **Кастомизируемый блок с информацией:** это опциональный блок, при желании можно оформить небольшое подобие резюме с основными данными и проектами. Для работы с функционалом есть полная поддержка Markdown и поддержка основных HTML-тегов. Если не оформлять эту часть, то на ее месте будет блок с популярными репозиториями пользователя.

На втором скриншоте (если пролистать страницу профиля ниже):

1. **Блок с закрепленными репозиториями:** GitHub позволяет закрепить на главной странице профиля до шести репозиториях, они будут сразу видны всем посетителям страницы.
2. **Количество вкладов:** в этом блоке можно увидеть количество вкладов за все время, а также за последний год, месяц и неделю. При нажатии на любой из периодов можно увидеть график с количеством коммитов за этот период.
3. **Описание деятельности:** в этом блоке доступно описание деятельности пользователя на GitHub. Например, сколько и в каких репозиториях было коммитов, сколько было открыто и закрыто запросов на изменения и т.д.

## Генерация открытого SSH-ключа

---

Git-сервисы используют аутентификацию по SSH-ключам. Это значит, что для того, чтобы работать с репозиторием, нужно сгенерировать SSH-ключ и добавить его в профиль на GitHub.

Сперва следует проверить нет ли уже сгенерированного SSH-ключа, который можно использовать. Процесс идентичен для всех ОС. Все пользовательские SSH-ключи по умолчанию хранятся в каталоге `~/.ssh`. Поэтому перейдем в него и проверим содержимое:

```
$ cd ~/.ssh
$ ls
known_hosts
```



Файлы `id_dsa` или `id_rsa` и есть то, что нам нужно. Файлы с расширением `.pub` — открытые ключи, остальные — приватные. Если файлов нет или даже нет каталога `.ssh` (в нашем примере каталог есть, а вот файлов нет), то все это можно сгенерировать.

Для этого можно воспользоваться утилитой `ssh-keygen`. На macOS и Linux она является частью пакета SSH, а на Windows поставляется вместе с `git`:

```
$ ssh-keygen -o
Generating public/private rsa key pair.
Enter file in which to save the key (/home/daniilshat/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/daniilshat/.ssh/id_rsa
Your public key has been saved in /home/daniilshat/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:Eep/BjjzpZzyiNhXNLTlLbmOQhTeAsg2HFigZ1+yku8 daniilshat@workstation
The key's randomart image is:
+---[RSA 3072]---+
|. *oo . |
|o * . . . . . |
|..o..oo+.+ o |
| o o =+.+.+ . |
| o o.=oS..o |
| o .*.=. |
| ....*oo |
| + .o+.o. |
| . E.... |
+-----[SHA256]-----+
```



Сперва система предложит директорию для сохранения по умолчанию, проверьте, чтобы было предложено сохранить в `.../.ssh/id_rsa`. Если вдруг будет что-то другое (это маловероятно), укажите верный путь. Далее утилита попросит ввести пароль для приватного ключа и повторить его. А потом выдаст сообщение об успешной генерации.

SSH-ключ выглядит вот таким образом:

```
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAgQDV5eqah0I12Dn+YZmsKZ70cjxvjN6TIcsGiKIcXHgxvQ+oM:
```



## Добавление SSH-ключа в GitHub

---

Далее необходимо добавить сгенерированный SSH-ключ в `github`. Для этого переходим в свой профиль. В правом верхнем углу кликаем на миниатюру со своей аватаркой и в открывшемся меню переходим в пункт «Settings».

► Скриншот (нажать, чтобы раскрыть)

На открывшейся странице переходим в «SSH and GPG keys».

► Скриншот (нажать, чтобы раскрыть)

Нажимаем на кнопку «New SSH key».

► Скриншот (нажать, чтобы раскрыть)

На открывшейся странице:

- В «Title» введите какое-то название ключа или его идентификатор. К примеру, название машины на которой вы создали ключ и собираетесь работать (Пример: Home MacBook Air);
- в поле «Key» вставьте сам сгенерированный ключ;
- Нажмите «Add SSH key».

► Скриншот (нажать, чтобы раскрыть)

Далее GitHub попросит ввести пароль от аккаунта и добавит ключ, а на почту придет уведомление о том, что все прошло успешно. Страница с ключами будет выглядеть вот так:

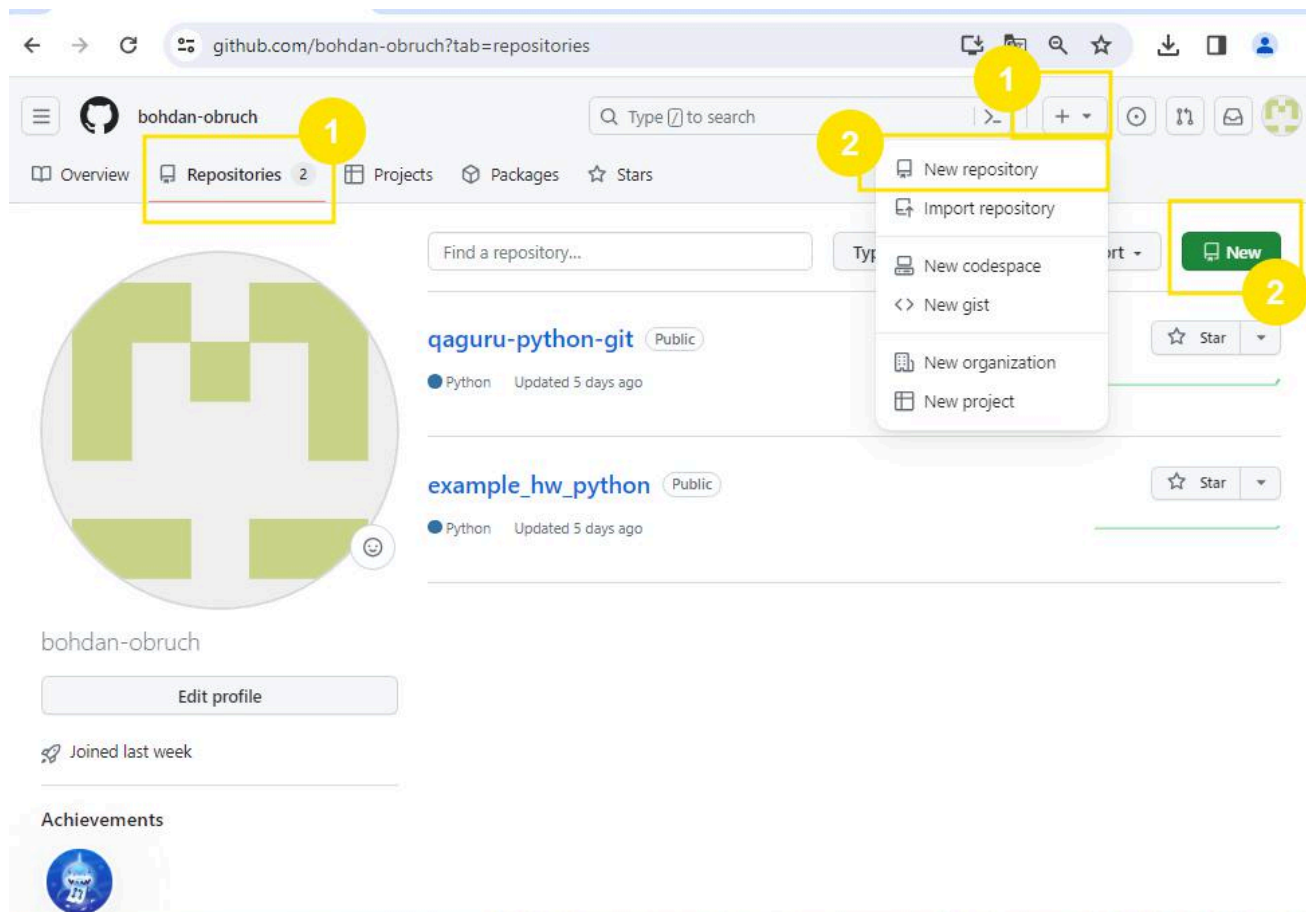
► Скриншот (нажать, чтобы раскрыть)

## Создание репозитория

---

Для создания репозитория есть 2 пути:

1. Перейти во вкладку `Repositories` и нажать на зеленую кнопку `New`.
2. В верхней част страницы кликнуть на иконку `+` и выбрать `New repository`.



На открывшейся странице необходимо:

► Скриншот (нажать, чтобы раскрыть)

1. Указать имя репозитория;
2. Добавить описание (необязательное поле);
3. Выбрать настройки приватности (публичный репозиторий или приватный);
4. Добавить README.md, .gitignore и выбрать лицензию. .gitignore можно сгенерировать в зависимости от используемого языка программирования. Для того чтобы создать именно для Python, необходимо кликнуть на gitignore template и выбрать Python из списка.
5. Нажать зеленую кнопку «Create repository».

## Алгоритм работы с Git / GitHub

### 1. Инициализация репозитория

Для инициализации репозитория необходимо в папке с проектом вызвать команду `git init`. В этот момент система создаст скрытую папку `.git` и будет сохранять в ней служебные файлы. Если ее удалить, то Git потеряет данные о репозитории и перестанет отслеживать изменения в проекте.

### Important

**Важно:** вызывать `git init` необходимо в папке с проектом.

## 2. Как добавить изменения

---

Для добавления изменений необходимо вызвать команду `git add <название файла>`. В этом случае Git добавит в индекс файл в текущем его состоянии. Если необходимо добавить не один файл, а все содержимое проекта, то следует вызвать `git add .`. Следует использовать внимательно и осторожно, иначе можно закоммитить много лишнего.

### Important

**Важно:** Индекс в Git — специальная промежуточная область, хранящая в себе изменения файлов на пути от рабочей папки до репозитория.

## 3. Как удалить изменения

---

Если в индекс попало лишнее, то такие файлы можно удалить из индекса с помощью команды `git rm --cached <название файла>`. Весь индекс сразу можно очистить с помощью `git rm --cached .`

## 4. Как не забивать репозиторий мусором (.gitignore)

---

Во время работы над проектом IDE и другие инструменты, создают много служебных файлов, которые не несут в себе практической пользы для разработчика. Их не надо коммитить в репозиторий и чтобы в ручную не отслеживать их состояние, можно указать список этих файлов в `.gitignore` -файле. Такой файл можно написать самостоятельно, а можно сгенерировать на [сайте](#).

## 5. Как контролировать статус файлов

---

Для контроля статуса всех файлов проекта можно воспользоваться командой `git status`. Система выведет список файлов в индексе, не отслеживаемых файлов и удаленных из индекса.

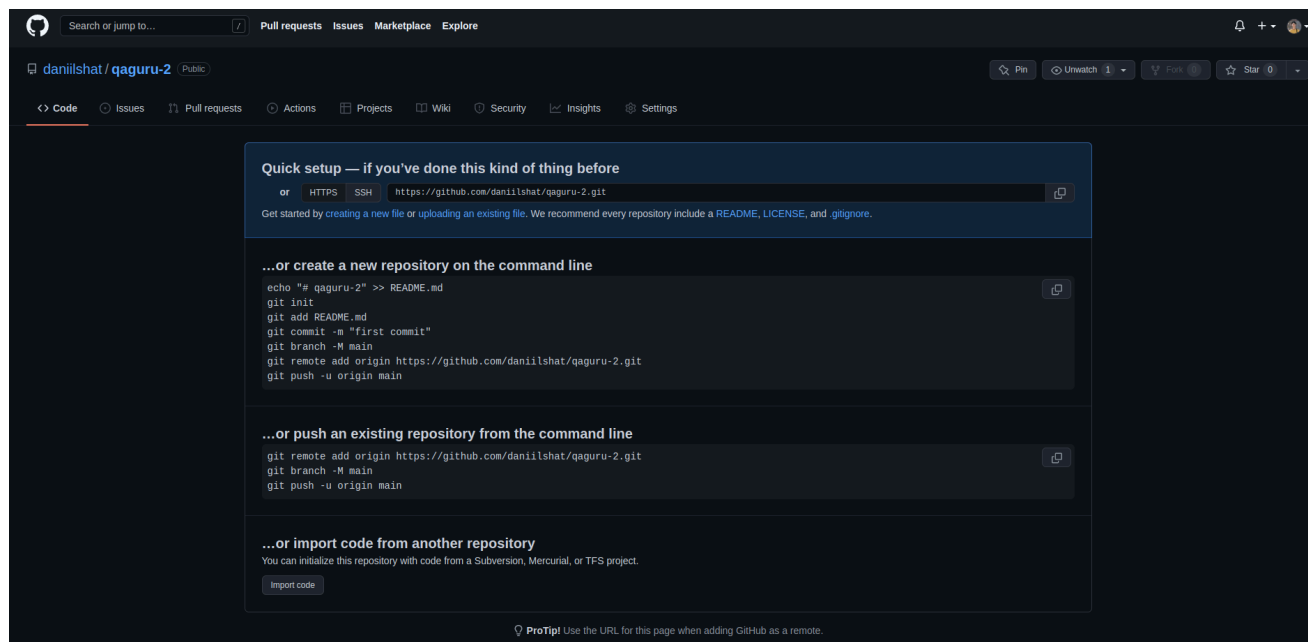
## 6. Как зафиксировать изменения в репозитории

В локальном репозитории изменения можно зафиксировать с помощью команды `git commit -m "Ваше сообщение"`. После вызова все файлы из индекса попадут в локальный репозиторий. Поэтому перед вызовом `git commit` необходимо выполнить `git add`. Также в строчке сообщений важно указывать осмысленные сообщения о внесенных изменениях.

## 7. Как подключить удаленный репозиторий GitHub

Сперва необходимо создать сам проект, проинициализировать в нем локальный репозиторий с помощью `git init` (смотри выше) и создать репозиторий на GitHub (смотри выше).

После создания пустого репозитория в GitHub на его странице увидим следующую инструкцию:



Если до этого все сделали правильно, то необходимо будет только выполнить команду `git remote add origin <SSH-репозитория или HTML-репозитория>`.

`origin` — псевдоним удаленного репозитория с помощью которого Git понимает, что мы собираемся вносить изменения именно в удаленный репозиторий. Он понадобится нам для выполнения команды `push`.

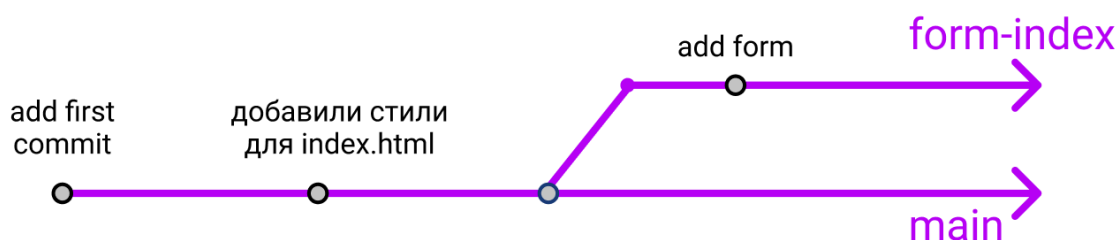


## 8. Как отправлять изменения в удаленный репозиторий GitHub

Для отправки изменения в удаленный репозиторий GitHub предусмотрена команда `push`. Если мы делаем первый пуш, то необходимо вызвать команду в следующем виде и обязательно с ключом `-u`: `git push -u origin <название ветки>`. Далее просто `git push origin <название ветки>`.

## 9. Работа с ветками

Ветки в Git помогают вносить изменения в код и не портить основной проект. Главная ветка с кодом основного проекта — `master` (иногда `main`, но это не так важно, оба названия указывают на то, что эта ветка главная). В любой момент времени от основной ветки можно создать новую с копией кода и продолжить работать в ней, а после слить с основной.



Основные команды:

- Создать новую ветку — `git branch <имя ветки>`
- Посмотреть список веток — `git branch`
- Переключиться на ветку — `git checkout <имя ветки>`

## 10. Слияния веток

Для слияния веток необходимо вызвать команду `git merge <имя сливаемой ветки>`. Разберем на примере. Условно у нас есть ветка `master` с основным проектом. Мы начали разрабатывать новую фичу и создали для работы ветку `feature`. Через некоторое время мы закончили и решили, что пора добавить наши изменения в основную ветку. Для этого необходимо перейти в ту ветку, в которую мы будем включать изменения и вызвать `git merge`.



```
$ git checkout master
Switched to branch 'master'
$ git merge feature
Merge made by the 'recursive' strategy.
```

## Pull request

GitHub не позволит нам просто так внести изменения и все испортить. Сперва наш код должны проверить. Для этого есть Pull request — механизм запросов на слияние ветки с `master`. После того как мы выполнили команду `merge` в основной ветки и слили свою с основной, в GitHub-репозитории появится запрос на создание Pull request. Нажимаем на зеленую кнопку.

### ► Скриншот (нажать, чтобы раскрыть)

Далее мы увидим страницу pull request, на ней есть:

1. Визуализация того, какие ветки предлагается слить;
2. Строка сообщения;
3. Поле подробного комментария;
4. Возможность пригласить других пользователей для ревью кода.

Заполняем все поля и нажимаем на зеленую кнопку «Create pull request».

### ► Скриншот (нажать, чтобы раскрыть)

На открывшейся странице мы можем посмотреть изменения в файлах (1) и одобрить слияние веток, если все в порядке (2):

### ► Скриншот (нажать, чтобы раскрыть)

После того как мы слили ветки, GitHub предложит нам удалить уже отработанную ветку, примем предложение:

### ► Скриншот (нажать, чтобы раскрыть)

## 11. Синхронизация локального репозитория с удаленным

Вероятно, что когда мы отдыхали, кто-то из команды работал и активно вносил изменения в главную ветку или наши ветки. Все эти изменения уже есть в удаленном репозитории, но теперь надо получить их и в нашем локальном репозитории. Для этих целей есть команда `git pull`.

## Работа с Git из IDE

# Основные моменты

---

## 1. Клонирование репозитория:

- Для того, чтобы клонировать репозиторий (например, из Github), необходимо на странице репозитория кликнуть на зеленую кнопку `Code` -> выбрать вкладку `HTTPS / SSH` в зависимости от способа клонирования репозитория и скопировать ссылку.

### ► Скриншот (нажать, чтобы раскрыть)

- Далее необходимо перейти в IDE, выбрать там `File -> New -> Project from Version Control` :

### ► Скриншот (нажать, чтобы раскрыть)

- В открывшемся окне вставить скопированный адрес в поле `URL` , выбрать директорию для размещения проекта и кликнуть на кнопку `Clone`

## 2. Отправка изменений в удаленный репозиторий:

- Для отправки изменений в удаленный репозиторий используется боковая панель слева:

### ► Скриншот (нажать, чтобы раскрыть)

Для отправки изменений в удаленный репозиторий необходимо:

1. "галочками" отметить те файлы, которые подлежат к отправке;
2. написать информативное сообщение, отображающее суть изменений;
3. кликнуть на кнопку `Commit and Push...` или `Commit` (только для фиксации изменений, но не для отправки)

## 3. Обновление состояния ветки (подгрузка изменений из удаленного репозитория):

1. На верхней панели, рядом с названием проекта кликнуть на название текущей ветки.
2. В списке кликнуть кликнуть на ту ветку, изменения в которую хотите "подгрузить"
3. В контекстном меню выбрать `Update`

### ► Скриншот (нажать, чтобы раскрыть)

## 4. Применение изменений из master (или другой) ветки в вашу ветку:

1. Предварительно обновить состояние веток, откуда и куда будут произведено применение изменений (см. п.3);
2. Переключиться( checkout ) на ту ветку, в которую планируется применение изменений;
3. Кликнуть на ветку, откуда хотите применить изменения;
4. Выбрать пункт merge {название ветки откуда применяете} into {название ветки в которую применяете}

► Скриншот (нажать, чтобы раскрыть)

## 5. Отслеживание изменений Git в IDE

В левом нижнем углу можно открыть панель Git, в которой можно множество полезного. Например, каждое зафиксированное изменение в различных ветках, с возможностью переключиться к каждой из них, сравнить с текущим состоянием ветки.

► Скриншот (нажать, чтобы раскрыть)

- ✓ [Сайт](#)
- ✓ [Чат в Telegram](#)
- ✓ [Канал в Telegram](#)

► Pages 100

[Содержание](#)  
[Командная строка](#)  
[Кодировка UTF-8 в Java](#)  
[Список полезных книг для автоматизаторов тестирования на языке Java](#)  
[Список полезных книг для автоматизаторов тестирования на языке Python](#) [Структура проекта Github](#)  
[README.md](#)

Java:

- [1. Вводное занятие. Сразу к практике.](#)
- [2. Git. GitHub. Погружаемся.](#)
- [3. Погружаемся в инструментарий и библиотеки](#)
- [4. Основы Java](#)
- [5. Продолжаем разрабатывать автотесты. PageObjects](#)
- [6. JUnit 5](#)
- [7. Allure Reports](#)
- [8. Работа с файлами](#)
- [9. Selenide #1](#)
- [10. Jenkins. Создаем первую задачу](#)
- [11. Управляем параметрами в коде и в Jenkins](#)
- [12. Отправляем уведомления о результатах прохождения автотестов](#)
- [13. Учимся быстро разрабатывать проекты для тестовых заданий](#)
- [14. Selenoid](#)
- [15. Библиотека Owner](#)
- [16. REST API. Пишем автотесты с Rest assured](#)
- [17. REST API. Декомпозируем UI тесты. Подключаем отчетность](#)
- [18. REST API. Продолжаем изучать](#)
- [19. Мобильная автоматизация #1. Разрабатываем автотесты с Browserstack](#)
- [20. Allure TestOps](#)
- [21. Переезд на собственную инфраструктуру Java](#)

Python:

- [1. Вводное занятие. Сразу к практике!](#)
- [2. Погружаемся в инструментарий и библиотеки](#)
- [3. Git. GitHub. Погружаемся.](#)
- [4. Основы Python](#)
- [5. Selene #1](#)
- [6. Основы Python. Часть II](#)
- [7. Основы Python. Часть III](#)
- [8. Page Object](#)
- [9. Allure Reports](#)
- [10. Работаем с файлами Python](#)
- [11. Jenkins. Создаем первую задачу и управляем параметрами Python](#)
- [12. Телеграм бот. Отправляем уведомления о результатах прохождения тестов](#)
- [13. Pytest](#)
- [14. Selenoid](#)
- [15. Pytest. Часть II](#)
- [16. Venv, Poetry и управление зависимостями проекта](#)
- [17. REST API. Часть I. Пишем автотесты с Requests](#)
- [18. REST API. Часть II. Продолжаем изучать](#)
- [19. Мобильная автоматизация #1. Разрабатываем автотесты с Browserstack](#)
- [20. Мобильная автоматизация #2. Разрабатываем автотесты с эмулятором Android устройства](#)
- [21. Allure TestOps](#)
- [22. Переезд на собственную инфраструктуру Python](#)

Clone this wiki locally

<https://github.com/qa-guru/knowledge-base/wiki.git>

